SPACECRAFT TELEMETRY COMPRESSION USING MACHINE LEARNING ASSISTED BEHAVIOUR MODELLING ON NASA CFS

A PROJECT REPORT

Submitted by

YASH VERMA [Reg No: RA2211019010023]

Under the guidance of

Dr. SENTHILKUMAR. S, Ph.D.
(Professor, Department of Aerospace Engineering)

and

PRASHANT KULSHRESHTHA
Division Head, Control & Mission Software Design Division, Controls System Group
U. R. Rao Satellite Centre, Bangalore

in partial fulfilment for the award of the degree of

BACHELOR OF TECHNOLOGY

in

AEROSPACE ENGINEERING

of

FACULTY OF ENGINEERING & TECHNOLOGY

SRM INSTITUTE OF SCIENCE & TECHNOLOGY
(Deemed to be University u/s 3 of UGC Act, 1956)

S.R.M. Nagar, Kattankulathur, Chengalpattu District

MAY 2026

SRM INSTITUTE OF SCIENCE & TECHNOLOGY

(Under Section 3 of UGC Act, 1956)

BONAFIDE CERTIFICATE

Certified that this project report titled "**SPACECRAFT TELEMETRY COMPRESION USING MACHINE LEARNING ASSISTED BEHAVIOUR MODELLING ON NASA CFS**" is the bonafide work of "**YASH VERMA**" who carried out the project work under my supervision. Certified further, that to the best of my knowledge the work reported herein does not form any other project report or dissertation on the basis of which a degree or award was conferred on an earlier occasion on this or any other candidate.

(Dr. SENTHILKUMAR. S)
INTERNAL GUIDE
Professor
Department of Aerospace Engineering

(Dr. L. R. GANAPATHY SUBRAMANIYAM)
HEAD OF DEPARTMENT
Department of Aerospace Engineering

(PRASHANT KULSHRESHTHA)
EXTERNAL GUIDE
Division Head
Control & Mission Software Design Division, Controls System Group
U. R. Rao Satellite Centre, Bangalore

Signature of the Internal Examiner Signature of the External Examiner

भारत सरकार
अन्तरिक्ष विभाग
यू.आर. राव उपग्रह केन्द्र
पोस्ट बॉक्स नं. १७९५, ओल्ड एयरपोर्ट रोड
विमानपुरा डाक, बेंगलूरु - ५६० ०१७, भारत
दूरभाष :
फैक्स :

इसरो isro

Government of India
Department of Space
U.R. RAO Satellite Centre
Post Box No. 1795, Old Airport Road
Vimanapura Post, Bengaluru - 560 017, India
Telephone :
Fax :

CERTIFICATE

This is to certify that **Mr. Yash Verma** (Reg No: RA2211019010023), IV Year, B.Tech (Aerospace Engineering) from **SRM Institute of Science and Technology, KTR**, has successfully completed Major Project on "**SPACECRAFT TELEMETRY COMPRESION USING MACHINE LEARNING ASSISTED BEHAVIOUR MODELLING ON NASA CFS**" in the Control & Mission Software Design Division under the Controls System Group at **U. R. Rao Satellite Centre**, Bangalore, under the guidance of **Prashant Kulshreshtha**, Division Head, from 7th January 2026 to 30th April 2026.

PRASHANT KULSHRESHTHA
Division Head
Control & Mission Software Design Division, Controls System Group
U. R. Rao Satellite Centre, Bangalore

ABSTRACT

This project presents the design, development and flight software integration of a machine learning assisted lossless telemetry compression system for a spacecraft operating under limited downlink bandwidth, along with a reconstruction algorithm. The objective is to maximize data throughput while ensuring strict lossless reconstruction, which is important for spacecraft telemetry.

A modular telemetry simulation algorithm was developed, supporting multiple correlation patterns and standardized formatting aligned with ISRO telemetry structures. Real telemetry was used throughout for validation. A behaviour-aware approach was adopted to categorize telemetry streams and guide adaptive compression. The resulting modular telemetry stream compression pipeline combines encoding and entropy-based techniques, designed for onboard deployment without reliance on external libraries. A corresponding ground based decompression pipeline reconstructs and verifies the lossless telemetry.

The onboard segment was integrated into NASA's Core Flight System (cFS) architecture using the cFS Basecamp framework. It runs the compression application and communication link, while the ground segment performs data reception, reconstruction, and validation. This integration shows practical usage within a flight software environment.

ACKNOWLEDGEMENT

I would like to show my sincere gratitude to my internal guide at SRM Institute of Science and Technology, **Dr. SENTHILKUMAR. S,** for his guidance, motivation, encouragement and support during this project.

I am also very thankful to my external guide at U. R. Rao Satellite Centre, **PRASHANT KULSHRESHTHA,** for giving me the chance to work under him in a highly enriching and technically challenging environment. His expert guidance, advice, constructive feedback, and constant encouragement were key to the project's success. I truly appreciate the time, effort, and knowledge shared by both my guides, which significantly contributed to my learning and overall experience.

I would also like to express gratitude to the Major Project coordinators for their support and cooperation and to my review panel for providing me with valuable feedback and support during the entire duration of the project.

TABLE OF CONTENTS

CHAPTER NO.	TITLE	PAGE NO.
	ABSTRACT	**iv**
	ACKNOWLEDGEMENT	**v**
	TABLE OF CONTENTS	**vi**
	LIST OF TABLES	**viii**
	LIST OF FIGURES	**ix**
	LIST OF SYMBOLS, ABBREVIATIONS AND NOMENCLATURE	**x**
1.	**INTRODUCTION**	**1**
	1.1 DATA COMPRESSION	1
	1.2 NATURE OF SPACECRAFT TELEMETRY	1
	1.3 ENCODING AND ENTROPY CODING	2
	1.4 ML ASSISTED BEHAVIOUR MODELLING	3
	1.5 NASA'S CORE FLIGHT SYSTEM (cFS)	3
2.	**LITERATURE REVIEW**	**4**
	2.1 ML IN SPACECRAFT TELEMETRY	4
	2.2 DATA COMPRESSION TECHNIQUES	5
	2.3 NASA cFS	5
3.	**OBJECTIVES AND METHODOLOGY**	**6**
	3.1 OBJECTIVES	6
	3.2 METHODOLOGY	7
	3.2.1 Telemetry Simulator	7
	3.2.2 Machine Learning	10
	3.2.3 Compression Pipeline	13
	3.2.4 Decompression Pipeline	21
	3.2.5 cFS Integration	21

4.	**DESIGN EVOLUTION AND EXPERIMENTAL EVALUATION**	**27**
	4.1 TELEMETRY SIMULATOR	27
	4.2 MACHINE LEARNING MODELS	28
	4.3 COMPRESSION METHODS	29
	4.4 cFS INTEGRATION	30
	4.5 SOFTWARE IMPLEMENTATION APPROACH	32
5.	**RESULT AND DISCUSSIONS**	**34**
	5.1 MACHINE LEARNING RESULTS	34
	5.2 COMPRESSION METHODS PERFORMANCE EVALUATION	35
	5.3 LOSSLESS VERIFICATION	38
	5.4 cFS RESULTS	38
6.	**CONCLUSION**	**40**
7.	**FUTURE ENHANCEMENTS**	**41**
8.	**REFERENCES**	**42**

LIST OF TABLES

S.No.	TITLE	PAGE NO.
5.1	Behaviour Mode Headers	35
5.2	Compression Methods Performance Evaluation	36
5.3	Compression for file-based telemetry	37

LIST OF FIGURES

S.No.	TITLE	PAGE NO.
3.1	System Flow	7
3.2	Telemetry Frames	8
3.3	Frame Structure	8
3.4	Mutation Concept	9
3.5	Single 256 Byte Frame	10
3.6	Dataset Labels	12
3.7	Compression Pipeline	20
3.8	Reconstruction Pipeline	21
3.9	Final cFS Pipeline	22
3.10	OBC Process	23
3.11	Packet (chunk) Receiver and Batch Rebuilder	24
3.12	Telemetry Batch Decoder and Reconstructor	24
3.13	Overall Pipeline	25
3.14	CAD model for CORTEX-SAT	26
4.1	AstroLab Orbit Propagator	28
4.2	ML Pipeline	28
4.3	Compression and Decompression Pipeline	30
4.4	cFS Integration Testing Version	30
4.5	cFS GUI	31
4.6	Verification Version of Decompressor	32
4.7	Testing Version Run with GUI	33
5.1	Resultant Tree After Training Model	34
5.2	Legend for Accepted and Rejected methods	35
5.3	cFS Target and Ground Running	39
5.4	CORTEX - SAT	39

LIST OF SYMBOLS, ABBREVIATIONS AND NOMENCLATURE

ABBREVIATION	FULL FORM
ISRO	Indian Space Research Organization
URSC	U. R. Rao Satellite Center
ISTRAC	ISRO Telemetry Tracking and Command Network
NASA	National Aeronautics and Space Administration
Frame ID	Frame Identification Number
ML	Machine Learning
cFS	NASA's Core Flight System
SB	Software Bus
cFE	Core Flight Executive
OSAL	Operating System Abstraction Layer
kb/s	Kilobits per second
F1	Feature 1
F2	Feature 2
ZRLE	Zero Run-Length Encoding
OBC	Onboard Computer
TCP	Transmission Control Protocol
CR	Compression Ratio
CORTEX - SAT	Compression Optimized Real-time Telemetry EXchange Satellite
kNN	k-Nearest Neighbours
CSV	Comma Separated Values file
GUI	Graphical User Interface
.txt	Text file
.bin	Binary file
LZ77	Lempel-Ziv 1977
LZW	Lempel-Ziv-Welch
CAD	Computer-aided Design

CHAPTER 1
INTRODUCTION

Spacecraft missions have certain bandwidth constraints for downlink, which limits the telemetry sent to the ground. It includes information about the health of the spacecraft, sensor readings, and the status of subsystems which must be sent reliably and without loss. This makes it very important to have fast, lossless telemetry data compression that can maximise data throughput while still being light enough to run onboard.

1.1 Data Compression

Data compression is the process of reducing the number of bits required to represent information. In spacecraft systems, compression is used to increase effective data throughput without modifying the communication hardware.

Compression techniques can be broadly classified as:

- Lossy compression, where some information is discarded (not acceptable for telemetry)
- Lossless compression, where the original data can be perfectly reconstructed

For spacecraft telemetry, lossless compression is mandatory, as even small data inaccuracies can lead to incorrect interpretation of system health or scientific results.

1.2 Nature of Spacecraft Telemetry

Spacecraft telemetry is a stream of hexadecimal data frames, independent of the physical meaning of individual parameters. While telemetry represents meaningful

quantities (temperature, voltage, etc.), compression systems operate on the data structure and statistical properties, not on semantic interpretation.

The spacecraft telemetry structure considered in this project consists of frames of hexadecimal data with a certain number of bytes per frame (256, 300, etc). During visualization of these frames, each byte is a 2 character hexadecimal representation, and these bytes are separated by a space character with every frame in a new line. Batches of these frames are called Master Frames, where each Master Frame consists of 32 frames.

Each frame consists of:

- Headers and metadata (timestamps, frame ID) bytes
- Payload bytes

In a telemetry stream, one frame is generated every 512 milliseconds. This forms time-stream data.

Telemetry streams usually have:

- Static data
- Sequential correlations (instant/slow varying data with some relation)
- Repeated correlations (data repeating every certain amount of time)
- Random/noisy data

1.3 Encoding and Entropy Coding

Encoding is the process of converting the data into a more efficient form by taking advantage of patterns. But if the data is truly random, encoding might not help and could even expand the data.

Entropy coding is a class of compression techniques that assign shorter representations to more frequent data patterns and longer representations to less frequent ones, based on their statistical probability. It is based on the concept of information entropy, which measures the randomness of data. Lower entropy means more compressibility.

1.4 Machine Learning Assisted Behaviour Modelling

Behaviour modelling refers to the process of identifying patterns in data and categorizing them based on their statistical or temporal characteristics. Machine Learning (ML) techniques enable automated identification of such patterns from data. By learning from historical telemetry, machine learning models can classify data into behavioural categories and guide the selection of appropriate encoding strategies.

1.5 NASA's Core Flight System (cFS)

NASA's Core Flight System (cFS) is a modular, reusable flight software framework designed for spacecraft onboard computers. It provides a standardized environment for developing and running flight applications.

Its key features are:

- Layered architecture enabling portability across hardware and operating systems
- Software Bus (SB) for message-based communication between applications
- Operating System Abstraction Layer (OSAL) for platform independence
- Core Flight Executive (cFE) providing essential services such as task management, timing, and event handling

It allows users to focus on their own software without worrying about integration complexities by letting them create cFS applications with their logic which easily integrate with the whole flight software.

CHAPTER 2
LITERATURE SURVEY

This Chapter contains a review of the existing research in telemetry and general data compression, machine learning applications in spacecraft systems and NASA cFS. The objective is to understand the various research and use the information for this project, which is a behaviour-aware lossless telemetry compression system in a flight software framework. The numbers refer to the list of references at the end of the report.

2.1 Machine Learning in Spacecraft Telemetry

The study on decision tree methods *Song and Lu (2015)* [12] provided a strong foundation for classification techniques. This along with more practical information from scikit-learn and the mathematical basics from *Deisenroth et al. (2020)* [1] were directly utilized in this project to design a decision-tree-based telemetry behaviour classifier.

Gao et al. (2012) [3] proposed an unsupervised anomaly detection method based on clustering of normal telemetry behaviour. The work shows how behavioural patterns in telemetry can be learned without labelled data. *Liang et al. (2021)* [4] investigated fault diagnosis using high-dimensional telemetry data. The work highlights the effectiveness of machine learning in extracting patterns from telemetry streams but relies on semantic interpretation of data. *Mukhachev et al. (2021)* [5] presented modern machine learning techniques for spacecraft health monitoring. The study emphasizes semantic understanding of telemetry. These approaches are focused on identifying semantic meaning of telemetry, which is not the case for this project.

2.2 Data Compression Techniques

Much research has been done on compressing telemetry data using both old and new data compression methods.

Arcangeli et al. (1992) [10] presented essential bit-level compression techniques for telemetry management and *Levenets (2018)* [7] introduced a system-oriented methodology for telemetry compression. The design of the compression pipeline used in this project was influenced by it. *Hung et al. (2013)* [8] contained model-based compression methodologies for sensor data. It was based on approximating or predicting sensor data, which is not the case in this project. This project utilizes machine learning strictly for behaviour modelling and not predicting future telemetry values. *Logeswaran and Eswaran (2001)* [9] performed a comparative analysis of various lossless compression algorithms. The study helped to choose the algorithms for this project's big compression benchmarking test. *Yu et al. (2020)* [6] suggested using different compression methods for different types of data, which is one of the main ideas behind this project.

Yamagiwa et al. (2020) [13] introduced the Adaptive Entropy Coding technique implementation and explanation. It was an introduction to entropy-based compression methods in this project. However, it was not used in the final version as no external software or libraries could be used in the onboard spacecraft segment of this project. *Dipperstein (2018)* [14-15] provided detailed explanations and implementations of Rice (Golomb) coding and Huffman coding. These resources were instrumental in understanding the mathematical foundations and implementation of entropy coding and directly contributed to the selection of final compression techniques.

2.3 NASA cFS

The Core Flight System (cFS) is a widely used flight software framework for spacecraft. Nasa's official documentation *Cudmore (2015)* [16], *NASA (2021)* [17] and *McComas (2012)* [18] provided the foundation and basic training for the software. *McComas (2025)* [20-22] and the GitHub repository for cFS Basecamp provided detailed documentation which was used to implement the final part of the project. Also, several online resources such as tutorials, documentation, and videos were used to gain practical understanding of cFS operation.

CHAPTER 3
OBJECTIVES AND METHODOLOGY

3.1 Objectives

The main objective of this project is to make a lossless telemetry compression system for a spacecraft to maximise data throughput.

The project has the following specific goals:

- To develop a modular telemetry simulator that can generate realistic spacecraft data with different correlation characteristics and a standard format.
- To develop a machine learning assisted behaviour modelling algorithm that classifies telemetry based on certain patterns and enables adaptive compression.
- To test and compare different encoding and entropy-based compression methods and to build a lightweight, fully custom pipeline that can be used onboard without needing any external libraries.
- To develop a corresponding decompression and reconstruction pipeline on the ground segment, making sure that strict lossless reconstruction and verification is done.
- To develop a complete communication pipeline between onboard and ground segments, including data transmission, reception, reconstruction, and validation.
- To gain practical understanding of spacecraft flight software architectures, particularly the NASA cFS framework.

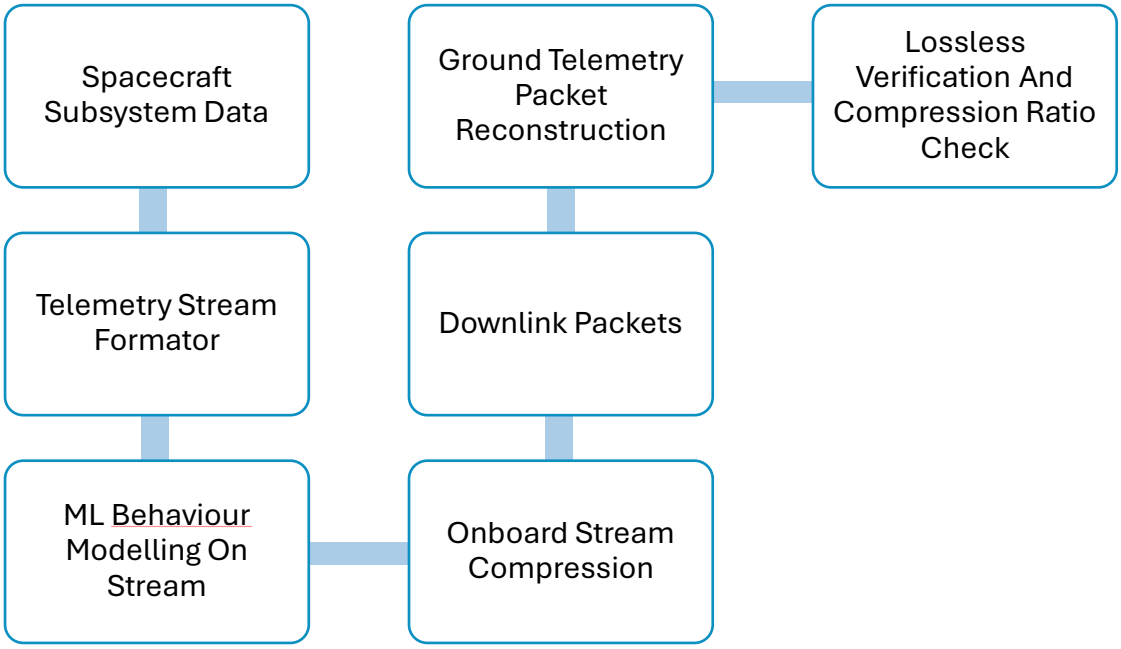

Figure 3. 1 System Flow

3.2 Methodology

All the sub-sections in this section contain the methodology for the final versions of the sub-systems of the project. All the work that led to these final versions is present in Chapter 4 of the report- 'Design Evolution and Experimental Evaluation'.

3.2.1 Telemetry Simulator

Telemetry is the data that is transferred from a spacecraft to the ground station. It is usually formatted as a series of 'frames' where each frame contains information regarding the spacecraft.

The telemetry frame format according to ISRO standards is in binary, but during visualization looks like a string of space separated bytes where each byte is represented as a hexadecimal value. In a single telemetry stream, every frame has the same number of bytes and is represented in a new line. Some common ones dealt with in this project are 256 bytes and 300 bytes per frame. Every frame contains a series of bytes which serves as a header. Headers tend to be constant throughout the stream but may change sometimes (like for every new master frame). Every frame also contains some bytes representing frame ID and timestamp. The remaining bytes represent the payload (power, health, sensor data etc.). A 'master

frame' is a set of 32 frames, often having similar data. Sometimes, a certain set of bytes repeats in a particular frame ID every master frame.

Figure 3.2 is an example of four 8-byte frames. In each frame, the first 4 bytes represent the header, the fifth byte represents the frame ID (00-1F cyclic), and the rest of the bytes represent the payload data.

```
FC CA 1D F9 00 0E 04 AF
FC CA 1D F9 01 0D 04 B5
FC CA 1D F9 02 0C 04 BC
FC CA 1D F9 03 0D 04 B7
```

Figure 3. 2 Telemetry Frames

The current standard frame generation rate (for a 256 byte frame telemetry stream) is 1 frame per 512 milliseconds. This comes up to be 4 kb/s (four kilobits per second). 4kb/s is the maximum downlink rate too. So, the goal of the project becomes to transmit much more than 4kb in one second, in that same 4kb/s transfer rate, by compressing the data.

The first part of the project consisted of making a telemetry simulator. The telemetry generated by this simulator would be used throughout the project for various purposes, including generating datasets, model training, and testing the compression pipelines. Only when each version of the compression pipeline was validated and confirmed to be working correctly, real ISRO telemetry was used.

A modular telemetry simulation algorithm was made which generates frames consisting of headers, frame IDs, and payload segments. Figure 3.3 is an example of the structure of a 256 byte frame.

| Header (5 bytes) | Frame ID (1 byte) | Payload (250 bytes) |

Figure 3. 3 Frame Structure

The payload is meant to mimic the different statistical patterns that can be seen in real telemetry streams. The modes include:

- constant or static data
- sequential data with low variation
- master-correlated patterns (where a pattern repeats across a master frame)
- random data

A mutation-based approach adds controlled randomness by changing a configurable number of bytes between each frame. Figure 3.4 shows a simplified way of mutation for the sequential or master correlated mode. Here, the number of random byte changes is configurable.

Let's say a telemetry stream starts from Frame 0. Then in sequential mode, "previous payload" would be the payload segment of Frame 0, and "new payload" would be the payload segment of Frame 1.

Previous Payload → Random Byte Changes → New Payload

Figure 3. 4 Mutation Concept

Similarly, in master correlation mode, "previous payload" would be the payload segment of Frame 0, but "new payload" would be the payload segment of Frame 32. As one master frame consists of 32 frames.

The simulator also works with mixed telemetry regimes, enabling the creation of datasets with different levels of correlation and randomness in a single stream. This allows to test how well compression works in a variety of situations.

The simulator only works with binary and hexadecimal data, without semantic meaning, which is in line with the design of the compression system. The generated telemetry is a binary stream but also is saved as a readable .txt file which can be used for verification of the reconstructed telemetry later.

```
F6 A5 3A F4 FC 00 FD 07 17 A3 AC 75 A2 F1 3D 14 B0 AB 63 84 B7 9E BE 9E 24 89 1A
DF DB D3 1B 39 84 94 8A 29 EF 0D A1 18 17 84 68 3F CE E1 E0 1C 0B 64 75 F5 E5 A8
89 45 B6 4C 36 34 70 F3 9B 3C 27 52 66 8F 36 71 44 8E F5 CD 76 F2 E3 35 09 E4 01
93 01 7F 97 2B B8 19 96 92 6A F8 C4 23 FC 31 5E F6 5D 72 FA BA 0F A8 3C 75 7B 6D
70 DC 8B D5 FB 73 46 36 F4 98 DE 5C 2A DC 22 B3 05 A9 75 CE 22 62 7C 9F 45 61 5E
34 87 D4 31 D4 58 DF 7C 3B 61 0D 1A 50 E5 A7 5B 00 2C A5 F0 CD F9 8E 15 30 3D A3
A9 99 95 BA E1 6E 11 D2 5B 3E 56 64 20 6F 88 34 C9 3A A4 29 B6 87 67 8C 28 D0 3A
CC 19 C6 5D 4F 2A 69 86 02 C2 ED E6 7F 3E 6B 86 01 B3 4F 2D BB 1D 83 C4 45 BC DC
24 92 64 9E 43 1A 53 34 0B 74 18 EF 8B 8A 2E 91 45 C2 27 AE 02 47 26 2A 0D AC 05
33 F9 D2 20 E1 C2 5D 9E ED 64 E9 AF BE
```

Figure 3. 5 Single 256 Byte Frame

3.2.2 Machine Learning

The machine learning component of this project was designed to enable behaviour-aware telemetry compression by classifying telemetry data based on statistical characteristics.

The ML pipeline consists of:

- Feature extraction from telemetry windows
- Dataset generation using compression-based labelling
- Training of a lightweight decision tree classifier
- Extraction of decision rules for onboard deployment

Telemetry is processed in fixed size windows of 32 frames ("Window" here is analogous to a master frame).

$$\text{Window} = \{\text{Frame}_1, \text{Frame}_2, \dots, \text{Frame}_{32}\} \qquad \rightarrow \textit{Eq 1}$$

Feature Extraction- A minimal and computationally efficient set of features was extracted from each telemetry window:

- **Feature 1: Sequential Zero Ratio (F1)**
  - ❖ Measures similarity between consecutive frames
  - ❖ High → sequentially related data

- **Feature 2: Master Zero Ratio (F2)**
  - Measures similarity with previous window (same frame index)
  - High → master-correlated data

$$F_1 = \frac{\text{Number of zero differences between consecutive frames}}{\text{Total number of bytes in a frame}} \quad \rightarrow Eq\ 2$$

$$F_2 = \frac{\text{Number of zero differences with previous window frame (same index)}}{\text{Total number of bytes in a frame}} \quad \rightarrow Eq\ 3$$

Now for **dataset generation,** first a decent amount of telemetry was generated using the telemetry simulator. This telemetry consists of mixed behaviour regimes. It is balanced across sequentially related, master-correlated and random data.

A set of 'features+label' datapoints for each window of the telemetry was made. These data points were used as a dataset to train the model. Instead of manually labelling data, a compression-driven labelling strategy was used :

For each window:

- Applied different compression strategies:
  - Sequential delta pipeline
  - Master delta pipeline
- Computed compression ratio
- Assigned label based on best performing method

So, for each window, two initial compression pipelines (a series of compression techniques in a particular order- explained in section 3.2.3) were applied. Only one process was changed in the pipeline- the process of Delta Encoding. First, it was done considering sequentially related frames, and then, considering master-correlated frames. The results of these two were considered to calculate

the compression ratios. The option with the lowest compression ratio was chosen as the label for the window. If the compression ratio indicated that the data has expanded instead of getting compressed (happens when the data is too noisy/random), it was given a 'bypass' label. Figure 3.6 shows the labels for these windows.

```
BYPASS=0
SEQUENTIAL=1
MASTER=2
```

Figure 3. 6 Dataset Labels

The datapoints were saved as:

$$[F1, F2, Label] \qquad \rightarrow Eq\ 4$$

Where each datapoint represented a window (32 frames) of telemetry. A set of more than 32,000 datapoints was used as the dataset.

The **Decision Tree Classifier** model was selected for final deployment due to:

- Low computational complexity
- Interpretability (rule extraction)

Training details:

- Stratified train-test split (80-20)
- Controlled depth (max depth = 3)
- Minimum samples per leaf for stability

After training the model, a decision tree containing threshold rules for the features was made:

- F1 > threshold → sequential
- F2 > threshold → master

The threshold values are given in Chapter 5 of the report. These threshold values enabled the segregation of telemetry windows into the three modes. This was the behaviour modelling part of the project. This will be used in the compression pipeline.

3.2.3 Compression Pipeline

The onboard compression system is designed as a real-time, behaviour-aware, lossless telemetry compression pipeline, operating on streaming telemetry data. The system processes telemetry in fixed size batches, applies adaptive encoding based on data behaviour, and produces compressed output suitable for transmission under constrained bandwidth.

The pipeline consists of:

- Behaviour-aware Delta Encoding
- Zero Run-Length Encoding (ZRLE)
- Entropy coding (One out of- Arithmetic coding, Adaptive-k Rice Encoding, Huffman Encoding)
- Adaptive bypass selection

Telemetry is processed in batches of multiple master frames (such as two or four master frames). This:

- Enables real-time operation
- Limits memory usage (as compression is happening on the spacecraft OBC)
- Allows independent compressed blocks

Initially, each raw telemetry frame has this structure:

$$\text{Frame} = \{\text{ Header, Frame ID, Payload }\} \equiv 256 \text{ continuous bytes} \quad \rightarrow Eq\ 5$$

First, for each batch for telemetry, features F1 and F2 are calculated for every master frame (32 frames) in these batches.

Then, the first process is **Delta Encoding**, where with the help of the thresholds given by the decision tree classifier model, master frames inside these telemetry batches are classifies into one of these modes(behaviours):

- Bypass (raw)
- Sequential delta
- Master-frame delta

Now, each frame looks like:

$$\text{Frame}_{\text{mode}} = \{\text{ Mode (1 byte) | Frame (256 bytes) }\} \quad \rightarrow Eq\ 6$$

Then the deltas are calculated according to the mode. Calculating a delta means subtracting corresponding bytes from the current frame and a reference frame. The first frame of a telemetry batch is sent raw (not subjected to delta encoding). These deltas are wrapped around 256 so that can be represented as one byte.

$$\Delta = (x_i - x_{ref}) \bmod 256 \quad \rightarrow Eq\ 7$$

Here x_i and x_{ref} are the corresponding (same index) current frame byte and reference frame byte respectively.

For sequential mode, the refence frame is the previous frame. And for master mode, the refence frame is the same frame ID in the previous master frame. Deltas aren't calculated for bypass mode. Frame is stored as is.

Now the frame structure looks like:

$$\text{Frame}_{\Delta} = \{\ \text{Mode (1 byte)} |\ \Delta_1, \Delta_2, \dots, \Delta_{256} (256\ bytes)\ \} \qquad \rightarrow Eq\ 8$$

And the stream looks like:

$$\text{Delta Stream} = \{\ \text{Frame}_{\Delta}^{(1)}, \text{Frame}_{\Delta}^{(2)}, \dots\ \} \qquad \rightarrow Eq\ 9$$

The second process is **Zero-Run Length Encoding (ZRLE).** After delta transformation, telemetry data contains a significant number of zero valued bytes, ZRLE encodes these zero sequences by replacing them with compact representations, reducing data size prior to entropy coding. The encoded stream consists of two types of packets:

- Zero Run Packet → stores count of consecutive zeros
- Literal Packet → stores non-zero byte sequences

Each packet is preceded by a tag byte indicating its type, followed by a length field and, in the case of literals, the actual data. A header containing the original size of the input stream is also included to enable accurate reconstruction during decompression .

The algorithm processes the input stream sequentially, maintaining buffers for zero runs and literal data. When a transition occurs between zero and literal regions, the corresponding buffer is flushed as a packet. This design ensures efficient handling of long zero sequences while preserving non-zero data without loss. Additionally, limits are imposed on packet sizes to ensure compatibility with fixed size length fields.

ZRLE is applied to the full batch of telemetry at once. So, the output is not frame-wise now, it's for the whole stream batch. Now the stream becomes:

$$\text{ZRLE Stream} = \{\ (0, L_z), (1, L_l, \text{data}), \ldots\ \} \qquad \rightarrow \textit{Eq 10}$$

Where 0 and 1 are the tags, L_z and L_l are the length of the field for the zeros and literals respectively and 'data' is the actual 'literal' bytes.

The third process is **Entropy Coding**, which after a major benchmarking experiment of multiple compression techniques (mentions is Chapter 4) narrowed down to Arithmetic Coding, Adaptive Mean-k Rice Coding and Huffman Coding. Only one of these three accepted methods are used in a pipeline at a time. Most of the time, Arithmetic Coding gave the best results while Adaptive Mean-k Rice Coding and Huffman Coding gave similar performance.

Arithmetic Coding (used as the primary entropy coding technique in the final pipeline) represents the entire input stream as a single fractional value within a continuously refined interval, unlike symbol-by-symbol encoding methods.

In this implementation, a frequency model is constructed for each batch by counting occurrences of all possible byte values (0-255). A cumulative distribution function is then generated and used to progressively narrow the encoding interval as each symbol is processed. The interval is updated using integer arithmetic with fixed precision, and scaling techniques are applied to maintain numerical stability. The encoded output is produced as a bitstream, ensuring efficient storage of the compressed data.

The encoder includes additional metadata such as the original data size and the frequency table, allowing the decoder to reconstruct the exact probability model.

$$high = low + \left\lfloor \frac{(high - low + 1)C(s+1)}{T} \right\rfloor - 1, \qquad \rightarrow Eq\ 11$$

$$low = low + \left\lfloor \frac{(high - low + 1)C(s)}{T} \right\rfloor \qquad \rightarrow Eq\ 12$$

Where,

$$T = \sum_{i=0}^{255} f(i) \qquad \rightarrow Eq\ 13$$

$$C(s) = \sum_{i<s} f(i) \qquad \rightarrow Eq\ 14$$

$$C(s+1) = \sum_{i \leq s} f(i) \qquad \rightarrow Eq\ 15$$

Where,

- low, $high$: current encoding interval bounds
- s: current symbol (byte value)
- $f(i)$: frequency of symbol
- $C(s)$: cumulative frequency up to symbol
- T: total frequency count (normalization factor)

Arithmetic coding encodes symbols by iteratively narrowing an interval based on cumulative symbol probabilities which is further based on symbol frequencies. For each symbol, the current interval is reduced proportionally to the symbol's probability range, resulting in a final interval that uniquely represents the entire data sequence.

After this, the stream looks like:

$$\text{Stream}_{\text{arith}} = \{ \text{Flags} \mid \text{Original Size} \mid \text{Frequency Table} \mid \text{Arithmetic Bitstream} \}$$

→Eq 16

Rice (Golomb) Coding is effective when the data distribution is skewed towards smaller values, which is typical for delta-encoded telemetry streams. In this method, each byte value is represented using a quotient-remainder decomposition controlled by a parameter k.

In this implementation, an adaptive mean based approach is used to dynamically estimate the optimal parameter k for each window of telemetry. For each window, the mean value of the data is computed. The parameter k is then estimated as a function of this mean, allowing the encoder to adapt to local variations in data distribution.

Each byte x is encoded in terms of:

- Quotient q encoded in unary (q ones followed by a zero)
- Remainder r encoded in binary (no. of bits=k)

$$x = q \cdot 2^k + r$$

→Eq 17

Where,

$$q = \left\lfloor \frac{x}{2^k} \right\rfloor$$

→Eq 18

$$r = x \bmod 2^k$$

→Eq 19

$$k = \lfloor \log_2 (\text{mean}(x)) \rfloor$$

→Eq 20

So, each encoded symbol looks like:

$$\text{encoded}(x) = \underbrace{11\cdots1}_{q \text{ times}}0 \mid\mid \text{binary}_k(r) \qquad \rightarrow Eq\ 21$$

The stream now looks like:

$$\text{Stream}_{\text{rice}} = \{\text{Original Size} \mid k \mid \text{Rice Bitstream}) \} \qquad \rightarrow Eq\ 22$$

Huffman Coding is a variable length entropy coding technique used to compress data by assigning shorter codes to more frequent symbols.

In this implementation, a frequency table is constructed by counting the occurrences of each byte value in the input stream . Based on these frequencies, a binary tree is built where nodes with the lowest frequencies are iteratively merged. Each leaf node of the tree represents a symbol, and the path from the root to the leaf defines its binary code. Left edges correspond to 0 and right edges to 1, ensuring that the resulting codes are prefix-free. The encoder replaces each symbol in the input stream with its corresponding code, producing a compressed bitstream.

$$L(s) \approx -\log_2 P(s) \qquad \rightarrow Eq\ 23$$

$$P(s) = \frac{f(s)}{\sum_i f(i)} \qquad \rightarrow Eq\ 24$$

Where,

- s: symbol (0-255)
- $P(s)$: probability of symbol
- $f(s)$: frequency of symbol
- $\sum_i f(i)$: total number of symbols in the stream
- $L(s)$: length of the Huffman code (in bits) assigned to symbol

Now the stream looks like:

$$\text{Stream}_{\text{Huffman}} = \{\ \text{Original Size} \mid \text{Frequency Table} \mid \text{Huffman Bitstream}\ \}$$

→Eq 25

Sometimes the size of a batch can increase instead of reducing (if the data is too random). So, for each batch, their sizes were compared after:

- Raw delta
- ZRLE output
- Entropy coded output

Then, the pipeline was stopped at the process that produced the smallest size for that batch, and a tag was added. This was the final compressed stream structure that is transmitted as a packet:

$$\text{Packet} = \{\ \text{Type(last pipeline process)}, \text{Length}, \textit{Encoded stream}\ \}$$

→Eq 26

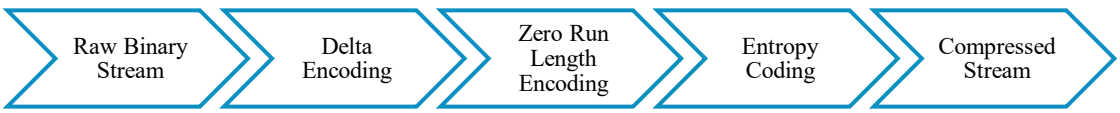

Figure 3. 7 Compression Pipeline

3.2.4 Decompression Pipeline

The decompression system performs the exact inverse operations of the onboard compression pipeline, ensuring bitwise lossless reconstruction of telemetry data. Each received packet was:

- Identified by type
- Decoded accordingly
- Reconstructed independently

For Arithmetic and Huffman coding methods, decoding was done by the stored frequency model in the headers. For Rice Coding, inverse of the math formulae was applied after navigating the unary and binary division for each symbol.

ZRLE decoding was done via expanding zero runs and restoring literal data. Using the stored mode headers, exact inverse of delta encoding was done to reconstruct each frame.

The reconstructed telemetry was compared to the original and bitwise validation ensured that the compression was lossless.

Figure 3.8 Reconstruction Pipeline

3.2.5 cFS Integration

The last step of the project is to use NASA's Core Flight System (cFS) to put the telemetry compression pipeline into a real flight software environment. This integration makes it possible for the compression algorithm to run in a modular, message-driven spacecraft software architecture, which connects the work of developing the algorithm with putting it into action onboard. The goal of this stage is not only to test the compression pipeline in a real-world setting, but also

to learn flight software design, structure applications, and communicate between modules.

The system flow is given in Figure 3.8.

The **onboard segment** is implemented on a **"target"**. Here the target, was a Raspberry Pi board acting as a spacecraft OBC. The following components were integrated within a cFS application:

- Telemetry simulator
- Behaviour-aware compression pipeline
- Data transmission interface

Telemetry frames are generated and processed in real time, passed through the compression pipeline, and transmitted to the ground system. The compression logic operates as part of the application layer, utilizing cFS services for execution and scheduling. Due to another goal of learning data transmission techniques, communication is established using a TCP-based link between the OBC and the ground instead of the native Software Bus downlink mechanism where cFS in on the ground system too.

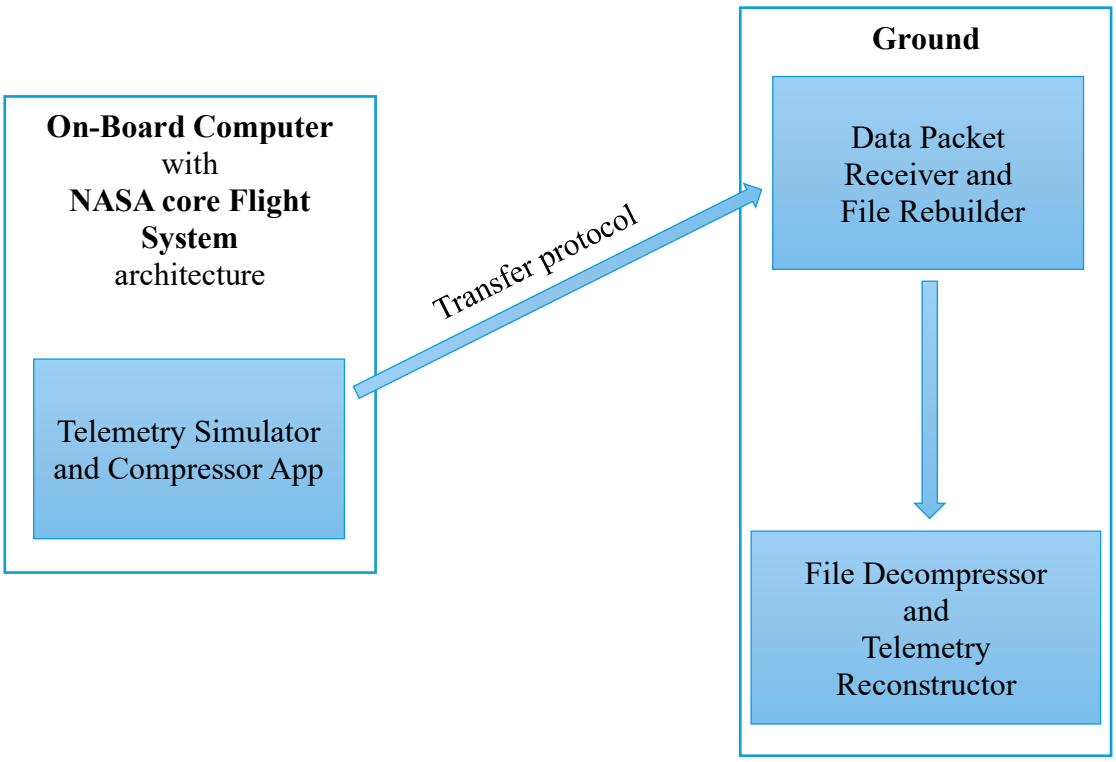

Figure 3. 9 Final cFS Pipeline

The generation and compression logic was embedded into the already existing app_c_demo_app in cFS Basecamp.

```
pi@raspberrypi: ~/CORTEX-SAT/cfs-basecamp/cfe-eds-framework/build/exe/cpu1
EVS Port1 66/1/APP_C_DEMO 996: CORTEX-SAT TX COMPRESSED - Sending idx=18480 / 19988, chunk=120
EVS Port1 66/1/APP_C_DEMO 996: CORTEX-SAT TX COMPRESSED - Sending idx=18600 / 19988, chunk=120
EVS Port1 66/1/APP_C_DEMO 996: CORTEX-SAT TX COMPRESSED - Sending idx=18720 / 19988, chunk=120
EVS Port1 66/1/APP_C_DEMO 996: CORTEX-SAT TX COMPRESSED - Sending idx=18840 / 19988, chunk=120
EVS Port1 66/1/APP_C_DEMO 996: CORTEX-SAT TX COMPRESSED - Sending idx=18960 / 19988, chunk=120
EVS Port1 66/1/APP_C_DEMO 996: CORTEX-SAT TX COMPRESSED - Sending idx=19080 / 19988, chunk=120
EVS Port1 66/1/APP_C_DEMO 996: CORTEX-SAT TX COMPRESSED - Sending idx=19200 / 19988, chunk=120
EVS Port1 66/1/APP_C_DEMO 996: CORTEX-SAT TX COMPRESSED - Sending idx=19320 / 19988, chunk=120
EVS Port1 66/1/APP_C_DEMO 996: CORTEX-SAT TX COMPRESSED - Sending idx=19440 / 19988, chunk=120
EVS Port1 66/1/APP_C_DEMO 996: CORTEX-SAT TX COMPRESSED - Sending idx=19560 / 19988, chunk=120
EVS Port1 66/1/APP_C_DEMO 996: CORTEX-SAT TX COMPRESSED - Sending idx=19680 / 19988, chunk=120
EVS Port1 66/1/APP_C_DEMO 996: CORTEX-SAT TX COMPRESSED - Sending idx=19800 / 19988, chunk=120
EVS Port1 66/1/APP_C_DEMO 996: CORTEX-SAT TX COMPRESSED - Sending idx=19920 / 19988, chunk=120
EVS Port1 66/1/APP_C_DEMO 996: CORTEX-SAT TX COMPRESSED - Sending idx=19988 / 19988, chunk=68
EVS Port1 66/1/APP_C_DEMO 996: CORTEX-SAT TX COMPRESSED - Sending idx=0 / 20123, chunk=0
EVS Port1 66/1/APP_C_DEMO 996: CORTEX-SAT TX COMPRESSED - Sending idx=120 / 20123, chunk=120
EVS Port1 66/1/APP_C_DEMO 996: CORTEX-SAT TX COMPRESSED - Sending idx=240 / 20123, chunk=120
EVS Port1 66/1/APP_C_DEMO 996: CORTEX-SAT TX COMPRESSED - Sending idx=360 / 20123, chunk=120
EVS Port1 66/1/APP_C_DEMO 996: CORTEX-SAT TX COMPRESSED - Sending idx=480 / 20123, chunk=120
EVS Port1 66/1/APP_C_DEMO 996: CORTEX-SAT TX COMPRESSED - Sending idx=600 / 20123, chunk=120
EVS Port1 66/1/APP_C_DEMO 996: CORTEX-SAT TX COMPRESSED - Sending idx=720 / 20123, chunk=120
EVS Port1 66/1/APP_C_DEMO 996: CORTEX-SAT TX COMPRESSED - Sending idx=840 / 20123, chunk=120
EVS Port1 66/1/APP_C_DEMO 996: CORTEX-SAT TX COMPRESSED - Sending idx=960 / 20123, chunk=120
EVS Port1 66/1/APP_C_DEMO 996: CORTEX-SAT TX COMPRESSED - Sending idx=1080 / 20123, chunk=120
EVS Port1 66/1/APP_C_DEMO 996: CORTEX-SAT TX COMPRESSED - Sending idx=1200 / 20123, chunk=120
```

Figure 3. 10 OBC process

On the OBC, when cFS is started, it generates and compresses the telemetry stream and transmits the packets in chunks. While doing this, it also prints messages (Figure 3.10) in the terminal showing the packet IDs that are sent along with their size.

In the **ground segment**, the system is implemented on a Linux environment and performs:

- Packet reception-Chunks received
- Telemetry compressed batch reconstruction- Chunks are joined to form compressed binary telemetry file
- Decompression and visual reconstruction- Decompression pipeline on the binary file (from .bin to .txt)

```
yash@yash-VirtualBox: ~
2026-04-07 02:06:18 [INFO] Batch 1 progress: 2040 / 20123 bytes
2026-04-07 02:06:19 [RX] Packet received: 132 bytes
2026-04-07 02:06:19 Telemetry chunk bytes: 120
2026-04-07 02:06:19 [INFO] Batch 1 progress: 2160 / 20123 bytes
2026-04-07 02:06:20 [RX] Packet received: 132 bytes
2026-04-07 02:06:20 Telemetry chunk bytes: 120
2026-04-07 02:06:20 [INFO] Batch 1 progress: 2280 / 20123 bytes
2026-04-07 02:06:21 [RX] Packet received: 132 bytes
2026-04-07 02:06:21 Telemetry chunk bytes: 120
2026-04-07 02:06:21 [INFO] Batch 1 progress: 2400 / 20123 bytes
2026-04-07 02:06:22 [RX] Packet received: 132 bytes
2026-04-07 02:06:22 Telemetry chunk bytes: 120
2026-04-07 02:06:22 [INFO] Batch 1 progress: 2520 / 20123 bytes
2026-04-07 02:06:23 [RX] Packet received: 132 bytes
2026-04-07 02:06:23 Telemetry chunk bytes: 120
2026-04-07 02:06:23 [INFO] Batch 1 progress: 2640 / 20123 bytes
```

```
yash@yash-VirtualBox: ~
2026-04-07 01:53:46 [RX] Packet received: 132 bytes
2026-04-07 01:53:46 Telemetry chunk bytes: 120
2026-04-07 01:53:46 [INFO] Batch 0 progress: 21720 / 21875 bytes
2026-04-07 01:53:47 [RX] Packet received: 132 bytes
2026-04-07 01:53:47 Telemetry chunk bytes: 120
2026-04-07 01:53:47 [INFO] Batch 0 progress: 21840 / 21875 bytes
2026-04-07 01:53:48 [RX] Packet received: 132 bytes
2026-04-07 01:53:48 Telemetry chunk bytes: 35

2026-04-07 01:53:48 Batch 0 received
2026-04-07 01:53:48 Size: 21875 bytes
2026-04-07 01:53:48 Master Frames: 4

2026-04-07 01:53:50 [RX] Packet received: 132 bytes
2026-04-07 01:53:50 Telemetry chunk bytes: 120
2026-04-07 01:53:50 [INFO] Batch 1 progress: 120 / 21990 bytes
```

Figure 3. 11 Packet (chunk) Receiver and Batch Rebuilder

```
yash@yash-VirtualBox: ~
yash@yash-VirtualBox:~$ python3 watch_decode_from_OBC.py
Listening in . for received_batch_*.bin for
NASA cFS Telemetry batches

=== NASA cFS Telemetrty batch detected ===
File: received_batch_0.bin
Type: Arithmetic
Compressed Telemetry batch size: 19988
Frames reconstructed: 128 = 4.0 Master Frames
Raw size: 32768
Reconstructed size: 32768
Compression ratio: 1.639
Status: Reconstruction SUCCESS
Decoded frames saved to: received_batch_0.decoded.txt
```

Figure 3. 12 Telemetry Batch Decoder and Reconstructor

The ground decoder also prints the:

- type of compressed batch (last compression method used)
- No. of frames in the batch
- Original size
- Compressed size
- Compression ratio
- Reconstruction status and output file

The lossless verification check happens in the testing version of cFS pipeline, which is explained in Chapter 4.

The cFS integration validates that the proposed compression system is not only theoretically effective but also practically deployable within a realistic spacecraft flight software environment.

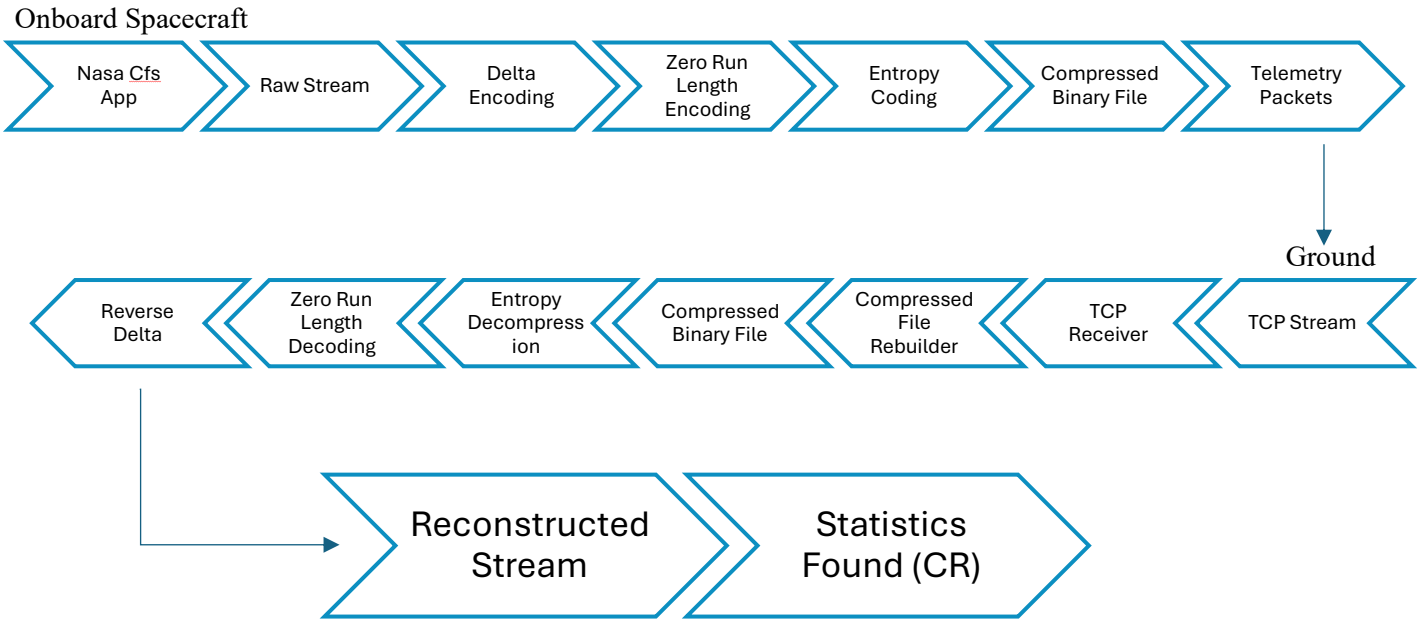

Figure 3. 13 Overall Pipeline

For demonstration purpose, the whole pipeline was integrated into an acting OBC target- a Raspberry Pi. A case was designed for this acting OBC in the shape of a satellite strictly for demonstration purposes and visual appeal. It has slots for the OBC, its power source and any sensors if used in the future. This “satellite” model along with the acting OBC which included the onboard segment pipeline was named:

Compression Optimized Real-time Telemetry EXchange Satellite (CORTEX - SAT).

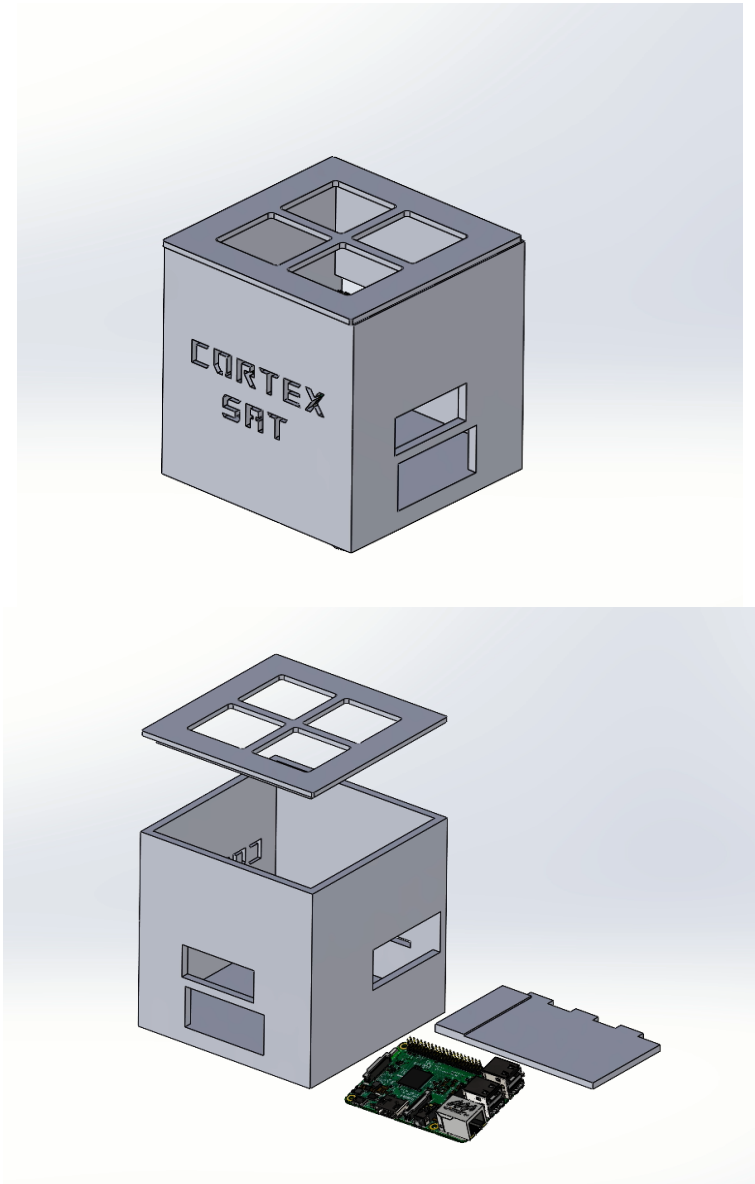

Figure 3. 14 CAD model for CORTEX-SAT

The ground segment is a Linux Environment (Ubuntu on VirtualBox) on a pc.

CHAPTER 4
DESIGN EVOLUTION AND EXPERIMENTAL EVALUATION

This Chapter contains the design evolution and experimental evaluation of the subsystems of this project. Each subsystem underwent multiple iterations before reaching the final implementation described in Chapter 3. The objective of this Chapter is to document this progression, proving brief explanation about each version.

4.1 Telemetry Simulator

Version 1 of the telemetry generator was developed to produce frames with a constant header, cyclic frame identifier, and payload values which incorporated simple relationships between two parameters, temperature and voltage, to simulate basic sensor behaviour. However, it was overly simplistic and not how real ISRO telemetry examples were like.

Version 2 had controlled variation in telemetry data by generating payloads based on previous frames with mutations. Modes of constant data and fully random data were also incorporated, which allowed mixed telemetry regimes.

Version 3 incorporated telemetry derived from orbital propagation data, introducing physical parameters such as position, velocity, orientation, and derived acceleration values. AstroLab-orbit propagator was used for the physical values which is an orbit propagator I developed for ISTRAC.

Version 4 is the final telemetry simulator incorporated in the stream generator mentioned in Chapter 3.

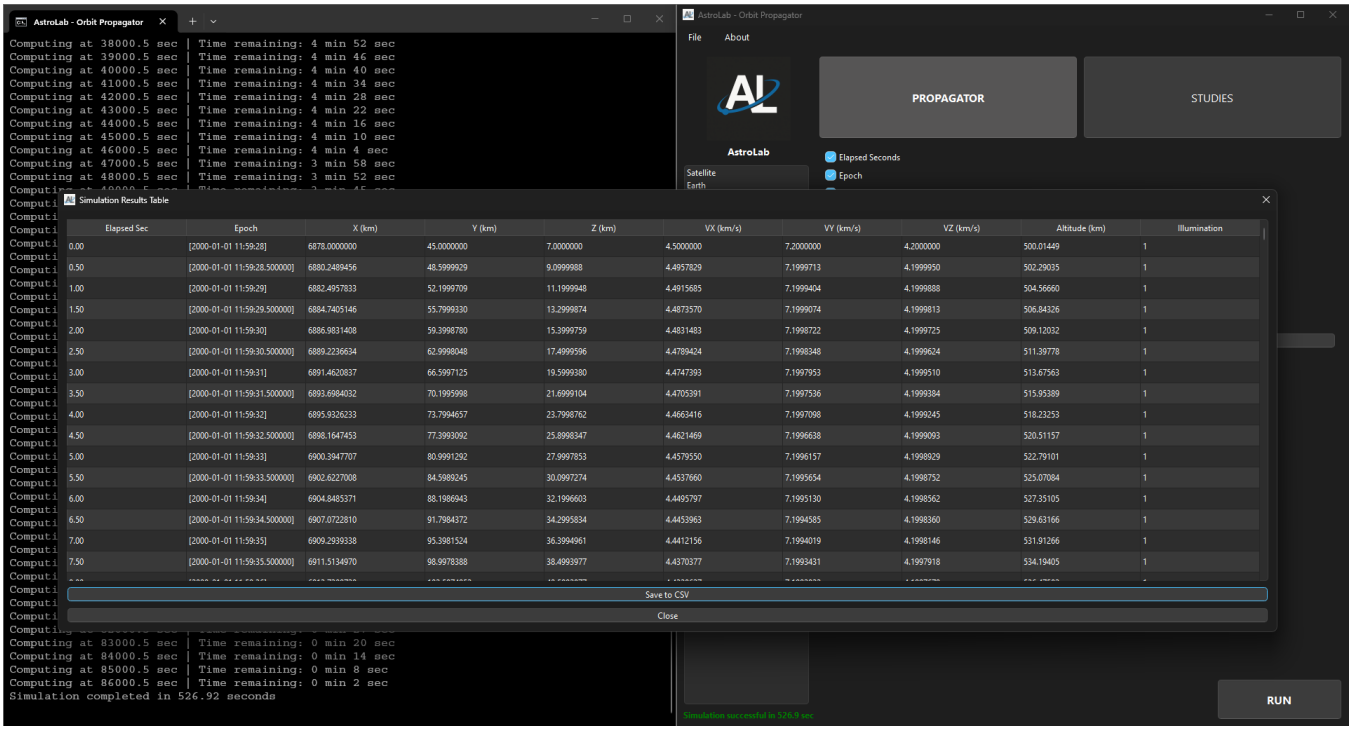

Figure 4.1 AstroLab Orbit Propagator

4.2 Machine Learning Models

Version 1 consisted of k-Nearest Neighbours (kNN) algorithm to identify patterns in telemetry data based on similarity between data points. Features such as temperature and voltage were extracted from telemetry frames and Euclidean distance was used to determine similarity between samples. However, this approach relied on storing entire datasets, making it unsuitable for real-time or onboard implementation.

Version 2 To shifted to a decision tree based classification model, which uses extracted statistical features to classify telemetry behaviour . The model was trained on a labelled dataset and provided simple threshold-based rules that could be directly implemented in the compression pipeline. This is explained in Chapter 3.

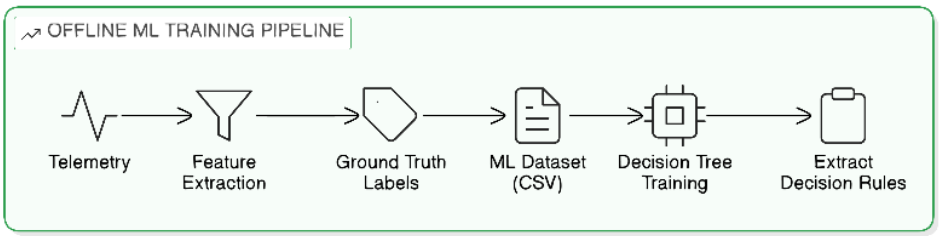

Figure 4.2 ML Pipeline

4.3 Compression Methods

Version 1 consisted of Adaptive Entropy Coding, which provided very good compression results, but could not be used due to it being an external software/library which was not allowed in the project.

Version 2 consisted of Rice encoding, which later went on to be one of the methods used in the final pipeline.

Then, a performance evaluation was done for the following compression methods, after learning in detail about the working of each method and implementing then on telemetry streams:

- Raw data
- Delta Encoding
- Delta of Delta Encoding
- Run Length Encoding
- Adaptive Entropy Coding
- Golomb Rice Encoding(fixed k)
- Adaptive Golomb Rice Encoding(mean k)
- Adaptive Golomb Rice Encoding(optimal k)
- Huffman coding
- Bitpacking
- LZ77
- LZW
- Arithmetic Coding

The final methods and combination of methods were chosen from the above based on the compression performance on the same telemetry. These methods are given in Chapter 3 and Chapter 5. The corresponding inverse process was used for decompression and reconstruction.

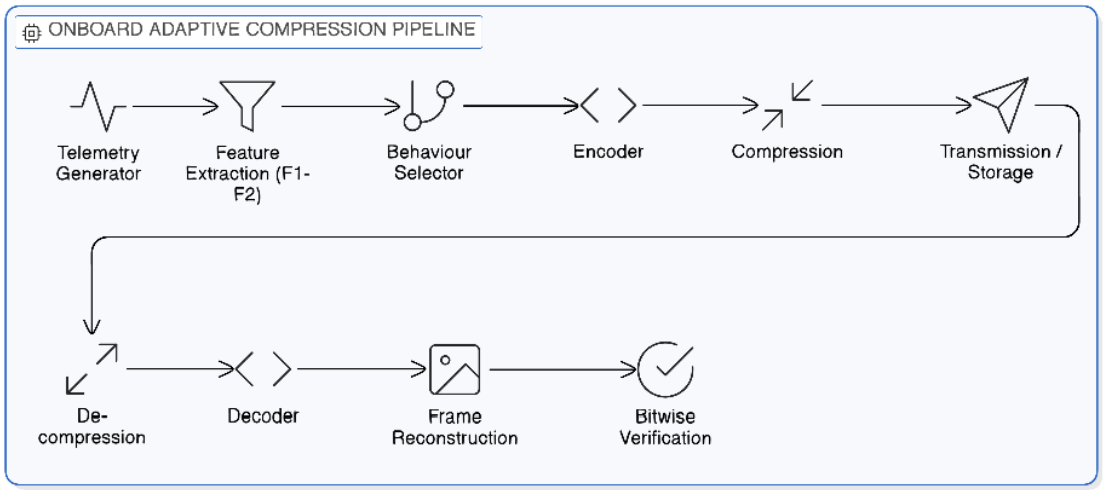

Figure 4.3 Compression and Decompression Pipeline

4.4 cFS Integration

Version 1 consisted of a same-system communication (local testing) where the target and the ground were the same system. So, the same local ip 127.0.0.1 was used for the transfer protocol at both ends. In this version, the GUI for cFS basecamp was also used to run and explore cFS. This was mainly for testing each component of the cFS integration before moving on to an actual OBC target and ground communication.

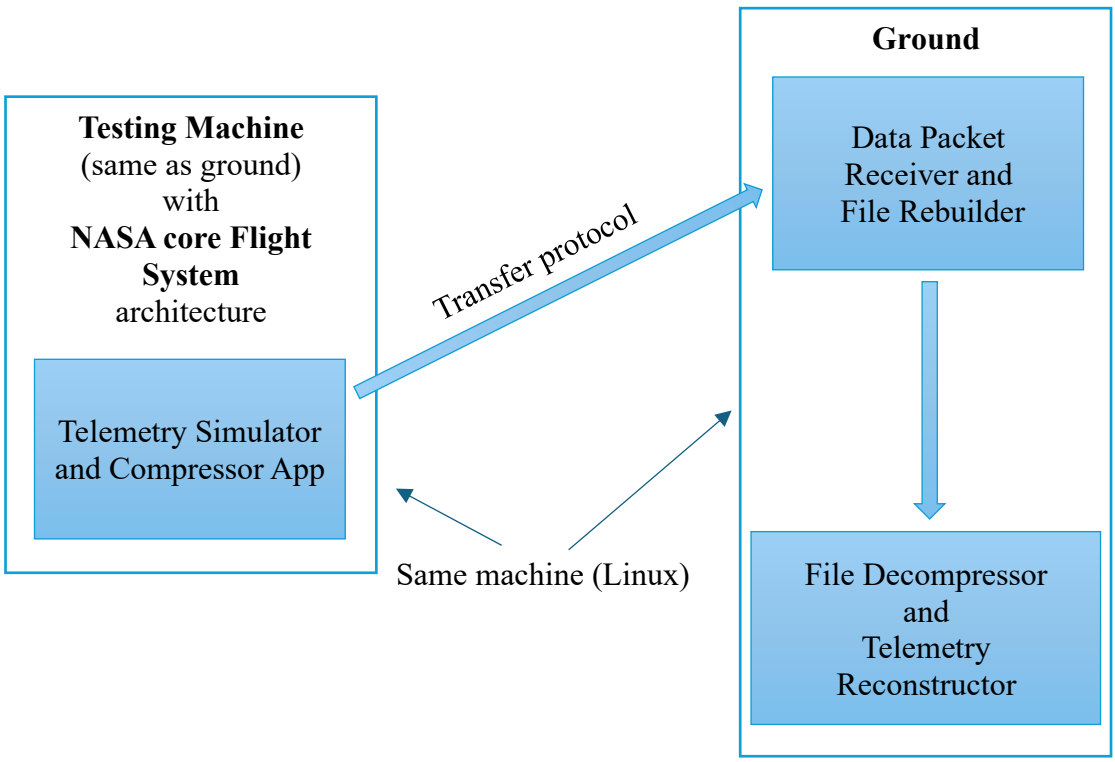

Figure 4.4 cFS Integration Testing Version

The GUI provided a visual interface for observing telemetry packets, application status, and system logs in real time, enabling easier debugging and validation of the compression application. It also allows manual command execution. While not part of the final deployed system, the GUI was particularly useful for development, testing, and verification of correct application behaviour within the cFS framework.

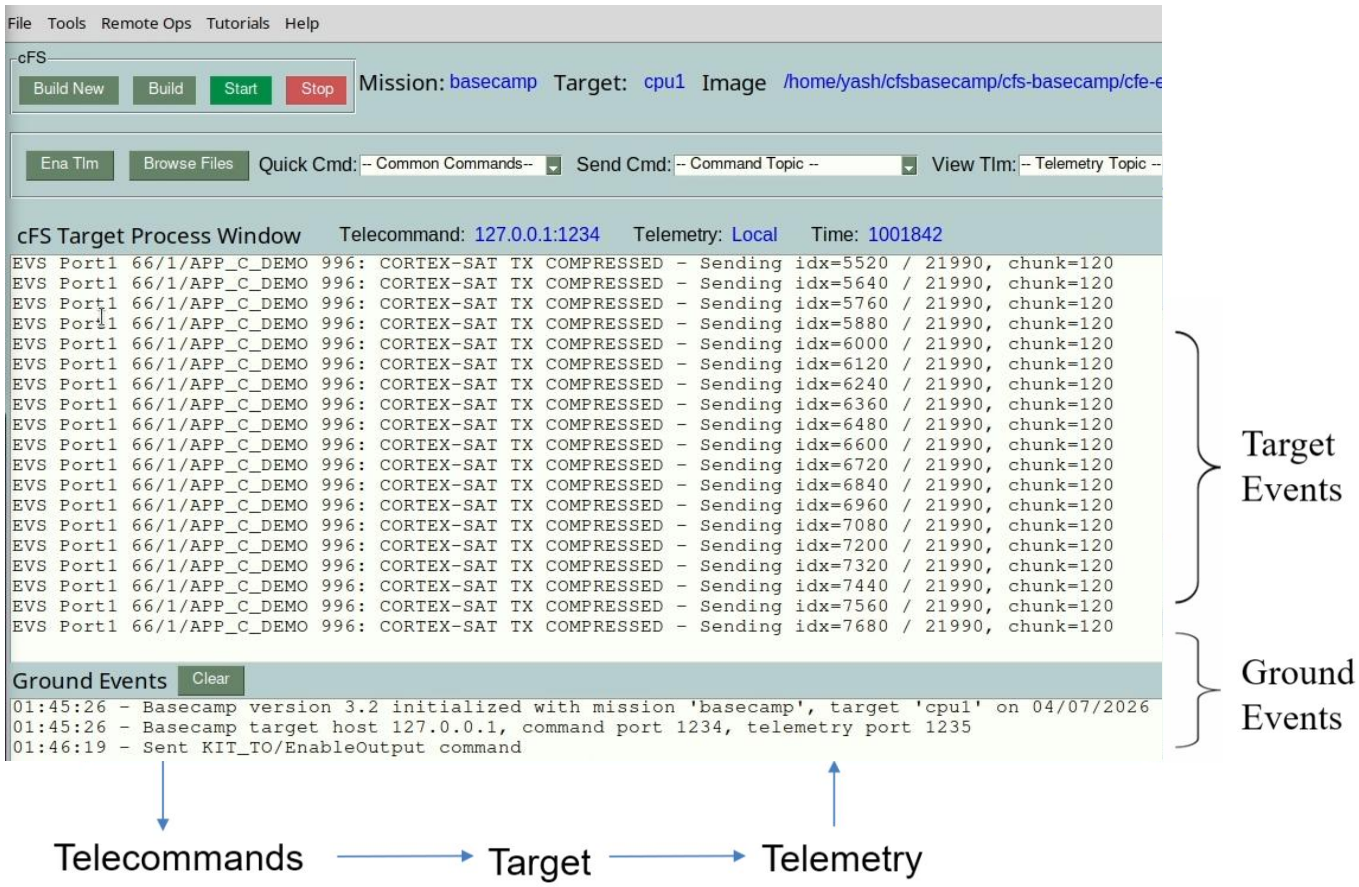

Figure 4.5 cFS GUI

In this version, the ground code responsible for decompressing and reconstructing the telemetry also had an additional feature to match the reconstructed telemetry with the original telemetry.

So, purely for testing, the first frame of a batch of telemetry was saved in a .txt file format every time a telemetry batch was generated on the target (with is the same machine as ground in this version). Once the whole the generation, compression, transmission, rebuilding and decompressing pipeline was completed, the reconstructed

telemetry batch's first frame was compared to the initial first frame text file saved during the generation process. Since the compression logic remained consistent throughout a batch, a match in the first frame would mean every corresponding frame also matches.

This is a verification process which ensures and lets the user know that the whole system is lossless and telemetry reconstructs to its exact initial form.

```
yash@yash-VirtualBox: ~
yash@yash-VirtualBox:~$ python3 watch_decode_vm.py
Listening in . for received_batch_*.bin
for NASA cFS Telemetry batches

=== NASA cFS Telemetrty batch detected ===
File: received_batch_0.bin
Type: Arithmetic
Compressed Telemetry batch size: 21875
Frames reconstructed: 128 = 4.0 Master Frames
Raw size: 32768
Reconstructed size: 32768
Compression ratio: 1.498
Status: Reconstruction SUCCESS
Decoded frames saved to: received_batch_0.decoded.txt
First frame check: MATCH
```

Figure 4.6 Verification Version of Decompressor

As seen in figure 4.6, 'First frame check: MATCH' suggests that the reconstructed telemetry batch is matching with the original generated telemetry batch.

The final version did not contain this feature, as finally the target was a spacecraft OBC and did not share storage with the ground. Testing with original telemetry would have to include transmitting the first frame text file along with the compressed binary packets, which defeats the whole point of this project. And since the testing version showed results where every single time the comparison was a match, the system is lossless.

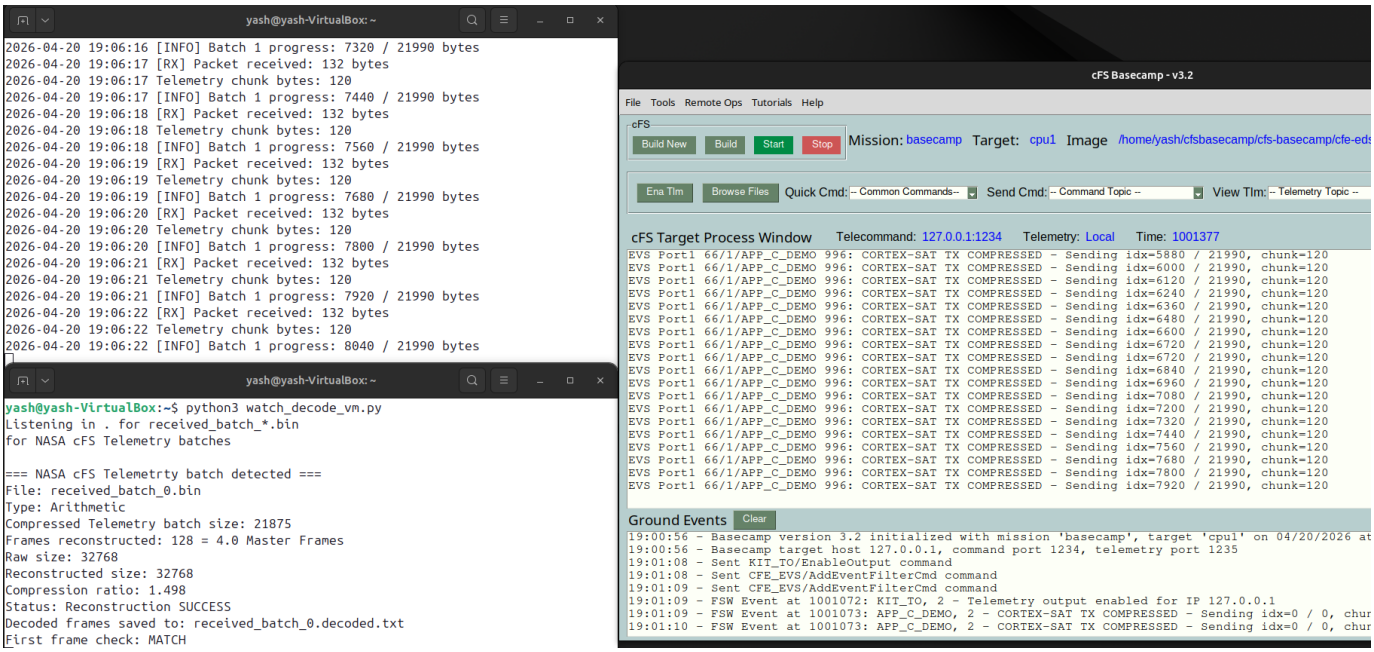

Figure 4.7 Testing Version Run with GUI

4.5 Software Implementation Approach

The development of all subsystems, including telemetry generation, machine learning, compression, and decompression pipelines, was initially carried out in Python.

Once the final pipeline was established, the core compression and related components were reimplemented in C for deployment while the ground modules remained in python.

This transition significantly improved execution speed and efficiency, making the system more suitable for real-time onboard applications. Additionally, since flight software development at URSC and the cFS framework itself are primarily C-based, this conversion ensured better compatibility and integration.

CHAPTER 5
RESULT AND DISCUSSIONS

5.1 Machine Learning Results

The trained decision tree classifier was used to derive simple rules for telemetry behaviour classification. These rules form the basis of what mode to choose for a particular master frame during delta encoding. The classification is based on the features representing sequential and master-frame correlations.

```
TREE RULES
|--- master_zero_ratio <= 0.87
|   |--- seq_zero_ratio <= 0.43
|   |   |--- master_zero_ratio <= 0.42
|   |   |   |--- class: 0
|   |   |--- master_zero_ratio >  0.42
|   |   |   |--- class: 2
|   |--- seq_zero_ratio >  0.43
|   |   |--- class: 1
```

Figure 5.1 Resultant Tree After Training Model

So, the following rules were considered when assigning a mode to a master frame in the initial process of the compression pipeline:

$$\text{Mode} = \begin{cases} 0 & \text{if } F_1 \leq 0.43 \wedge F_2 \leq 0.42 \\ 1 & \text{if } F_1 > 0.43 \\ 2 & \text{otherwise} \end{cases} \qquad \rightarrow \textit{Eq 27}$$

Where,

Mode	Behaviour
0	BYPASS
1	SEQUENTIAL
2	MASTER

Table 5. 1 Behaviour Mode Headers

5.2 Compression Methods Performance Evaluation

As mentioned in Chapter 4, all the compression methods were evaluated on a telemetry file with a series of different combinations. The comparison contains:

- Telemetry type
- Compression method
- .txt file size
- Raw telemetry binary file size
- Compressed file size
- Compression ratio 1 and saved space % (when compared to .txt)
- Compression ratio 2 and saved space % (when compared to .bin)

This is the key for the table 5.2:

not compressed/expanded
external library/software needed
rejected as final compression method is better in other combination
best combination of processes
not used as other methods provide better compression

Figure 5. 2 Legend for Accepted and Rejected methods

Telemetery type	method	TXT Size(B)	BIN(B)	compressed(B)	CR1	% saved(1)	CR2	% saved(2)
Raw TXT converted to Binary	aec	2,645,360	880,640	920,531	2.873733	65.202052	0.95666523	-4.529773801
	rice_fixed_k4	2,645,360	880,640	1,367,733	1.934120	48.296905	0.64386836	-55.31125091
	rice_adaptive_mean	2,645,360	880,640	933,874	2.832673	64.697659	0.94299659	-6.044921875
	rice_adaptive_best	2,645,360	880,640	933,803	2.832889	64.700343	0.94306829	-6.036859557
	huffman	2,645,360	880,640	881,668	3.000404	66.671153	0.99883403	-0.116733285
	rle	2,645,360	880,640	907,054	2.916431	65.711510	0.97087935	-2.99940952
	bitpack	2,645,360	880,640	881,289	3.001694	66.685479	0.99926358	-0.073696403
	lz77	2,645,360	880,640	983,772	2.688997	62.811413	0.89516677	-11.71102834
	lzw	2,645,360	880,640	1,240,820	2.131945	53.094475	0.70972421	-40.89980015
	arithmetic	2,645,360	880,640	880,558	3.004186	66.713113	1.00009312	0.00931141
Delta Encoded-Binary	aec	2,645,360	880,640	428,505	6.173464	83.801638	2.05514521	51.3416379
	rice_fixed_k4	2,645,360	880,640	783,265	3.377350	70.390986	1.12431936	11.05729924
	rice_adaptive_mean	2,645,360	880,640	561,643	4.710038	78.768750	1.56797111	36.22331486
	rice_adaptive_best	2,645,360	880,640	561,635	4.710105	78.769052	1.56799345	36.22422329
	huffman	2,645,360	880,640	358,502	7.378927	86.447894	2.45644376	59.2907431
	rle	2,645,360	880,640	455,992	5.801330	82.762573	1.93126195	48.22038517
	bitpack	2,645,360	880,640	863,228	3.064497	67.368222	1.0201708	1.977198401
	lz77	2,645,360	880,640	602,452	4.390989	77.226086	1.46175961	31.58929869
	lzw	2,645,360	880,640	668,380	3.957858	74.733874	1.31757384	24.10292515
	arithmetic	2,645,360	880,640	342,375	7.726499	87.057527	2.57215042	61.12202489
Delta of Delta Encoded-Binary	aec	2,645,360	880,640	533,339	4.959997	79.838699	1.65118246	39.43734102
	rice_fixed_k4	2,645,360	880,640	854,859	3.094499	67.684587	1.03015819	2.927529978
	rice_adaptive_mean	2,645,360	880,640	640,361	4.131045	75.793049	1.37522429	27.28458848
	rice_adaptive_best	2,645,360	880,640	640,340	4.131180	75.793843	1.37526939	27.28697311
	huffman	2,645,360	880,640	433,156	6.107176	83.625820	2.03307815	50.81349927
	rle	2,645,360	880,640	608,474	4.347532	76.998443	1.44729274	30.90547783
	bitpack	2,645,360	880,640	863,228	3.064497	67.368222	1.0201708	1.977198401
	lz77	2,645,360	880,640	736,380	3.592384	72.163335	1.19590429	16.38126817
	lzw	2,645,360	880,640	847,412	3.121693	67.966099	1.03921115	3.773164971
	arithmetic	2,645,360	880,640	427,104	6.193714	83.854598	2.06188657	51.50072674
RLE + Delta-Binary	aec	2,645,360	880,640	435,780	6.070402	83.526628	2.0208362	50.51553416
	rice_fixed_k4	2,645,360	880,640	517,971	5.107159	80.419640	1.7001724	41.1824355
	rice_adaptive_mean	2,645,360	880,640	416,876	6.345676	84.241237	2.11247469	**52.6621548**
	rice_adaptive_best	2,645,360	880,640	416,875	6.345691	**84.241275**	2.11247976	**52.66226835**
	huffman	2,645,360	880,640	346,250	7.640029	**86.911044**	2.54336462	**60.682004**
	bitpack	2,645,360	880,640	456,332	5.797007	82.749720	1.92982302	48.18177689
	lz77	2,645,360	880,640	527,384	5.016004	80.063810	1.66982692	40.11355378
	lzw	2,645,360	880,640	709,428	3.728863	73.182176	1.24133809	19.44176962
	arithmetic	2,645,360	880,640	344,813	7.671869	86.965366	2.55396403	**60.84518078**
RLE + Delta of Delta-Binary	aec	2,645,360	880,640	584,634	4.524814	77.899643	1.50630993	33.61259993
	rice_fixed_k4	2,645,360	880,640	683,829	3.868452	74.149870	1.28780733	22.34863281
	rice_adaptive_mean	2,645,360	880,640	554,307	4.772373	79.046066	1.58872249	37.05634539
	rice_adaptive_best	2,645,360	880,640	554,307	4.772373	79.046066	1.58872249	37.05634539
	huffman	2,645,360	880,640	456,343	5.796858	82.749304	1.92977651	48.1805278
	bitpack	2,645,360	880,640	608,925	4.344312	76.981394	1.4462208	30.85426508
	lz77	2,645,360	880,640	702,700	3.764555	73.436508	1.25322328	20.20575945
	lzw	2,645,360	880,640	899,364	2.941367	66.002208	0.97918084	-2.126180959
	arithmetic	2,645,360	880,640	454,210	5.824090	82.829936	1.93883886	48.42273801

Table 5. 2 Compression Methods Performance Evaluation

Figure 5.2 contains the key to see which all methods and combinations were selected and rejected from tables 5.2, along with the reason for the same.

The final method and combinations selected led to the following compression pipeline:

- Delta Encoding
- ZRLE
- Entropy Coding
    - Arithmetic Coding
    - Rice Coding
    - Huffman Coding

One interesting thing noticed was, the compression ratios achieved were greater when the compression pipeline was applied to telemetry files instead of streams, since more patterns could be picked up, and headers were stored only once for the whole file instead of every batch.

For a telemetry file that contains 172800 frames (1 day of telemetry) compression:

Step	Process	Size
0)	Telemetry raw file size (.bin)	43 MB
1)	After Delta Encoding (.bin)	43 MB
2)	After Zero RLE	16 MB
3) OR	After Rice Encoding	13 MB
3) OR	After Huffman Coding	9MB
3)	After Arithmetic Coding	9MB

Table 5. 3 Compression for file-based telemetry

5.3 Lossless Verification

The decompression pipeline successfully reconstructed the original telemetry data for all test cases. Bitwise comparison between the original and reconstructed data confirmed the correctness of the compression system and verified its strictly lossless nature.

In file based testing, the reconstructed file for directly compared to the original .txt telemetry file.

In telemetry stream testing, as soon as every batch in the stream went through the compression pipeline, it was also converted to .txt and saved for comparison with the reconstructed telemetry.

In initial cFS pipeline testing, where the target and ground were the same system, a part of the original telemetry batch was saved as a .txt file along with transmission of the compressed batch of the stream. This allowed comparison of files and verification of lossless compression and reconstruction.

Once confirmed to be lossless across all test cases and all types of telemetry, the final pipeline was incorporated with an actual spacecraft OBC as the target and only the compressed binary batches of the stream were transmitted.

5.4 cFS Results

The compression system was successfully integrated and executed as an application within NASA cFS Basecamp in the CORTEX-SAT. The onboard application processed telemetry data in real time and transmitted compressed data to the ground segment through a TCP-based communication link.

The ground system successfully received, reconstructed, decompressed and saved the telemetry stream as batches.

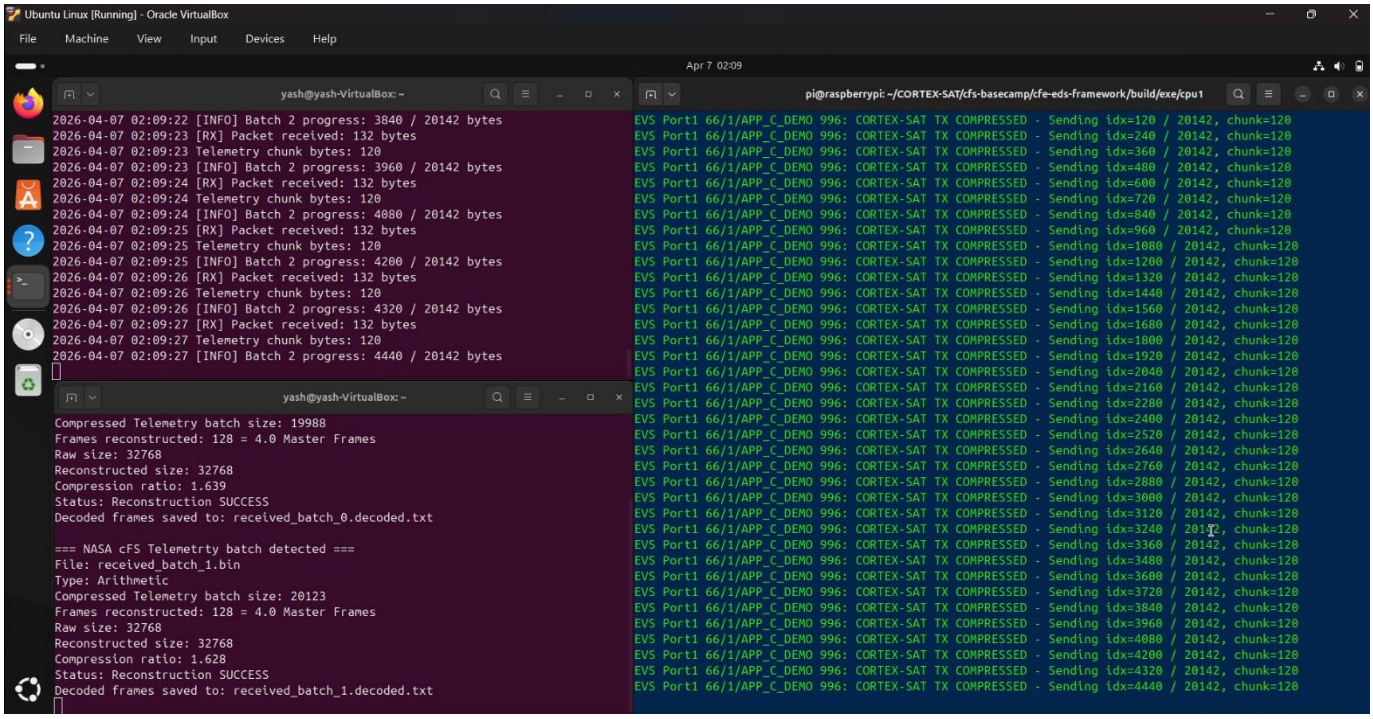

Figure 5. 3 cFS Target and Ground Running

In Figure 5.3, the right-side (blue) is the cFS application running on the target (OBC) while the left side (purple) is the Linux-based ground receiving and reconstructing telemetry.

The CORTEX-SAT was also successfully 3D printed using PLA material as shown in figure 5.4.

Figure 5. 4 CORTEX - SAT

CHAPTER 6
CONCLUSION

This project presented the design, development, and implementation of a machine learning-assisted lossless telemetry compression system for spacecraft operating under constrained communication bandwidth. The objective was to maximize effective data throughput by data compression while ensuring lossless telemetry transmission and reconstruction.

A behaviour-aware approach was used to classify telemetry based on statistical properties and not semantic meaning. A machine learning model- Decision Tree was used to find different data behaviours and extract feature thresholds. These features were used in the modular compression pipeline, which consisted of Delta Encoding, Zero Run-Length Encoding, and three methods of Entropy Coding over batches of a telemetry stream. The system was implemented without external libraries, ensuring suitability for deployment onboard spacecraft. A ground decompression pipeline enabled full reconstruction and bitwise verification of telemetry data, confirming lossless performance. The system was integrated into NASA's Core Flight System (cFS), with the onboard segment handling telemetry simulation and compression, and the ground segment handling reception and reconstruction. This demonstrated compatibility with real flight software architectures and provided practical insight into modular and real-time system design.

The project successfully bridges the gap between data-driven compression techniques and practical flight software implementation, providing a foundation for future intelligent onboard data handling systems.

CHAPTER 7
FUTURE ENHANCEMENTS

A few enhancements that can be explored to further improve performance and scalability of the project are:

- Onboard Machine Learning Integration:

  In the current implementation, machine learning is used offline to derive decision rules or thresholds for behaviour modelling. Further, an onboard ML model capable of adapting in real time to changing telemetry patterns by identifying patterns or by changing the amount of telemetry per batch, can be implemented. This could prove to show better compression results, and as a result, more data throughput.

- Full cFS-Based Ground Segment:

  The present system uses a custom ground-side implementation for data reception and decompression. Future work can involve deploying cFS on the ground segment in addition to the onboard segment, enabling communication through the Software Bus instead of external protocols. This would create a fully integrated end-to-end cFS system.

REFERENCES

[1] Deisenroth, M. P., Faisal, A. A. and Ong, C. S. (2020) Mathematics for Machine Learning, Cambridge University Press.

[2] Larrakoetxea, N. G., et al. (2023) 'Efficient Machine Learning on Edge Computing Through Data Compression Techniques', IEEE Access.

[3] Gao, Y., Yang, T., Xu, M. and Xing, N. (2012) 'An Unsupervised Anomaly Detection Approach for Spacecraft Based on Normal Behavior Clustering', Intelligent Computation Technology and Automation.

[4] Liang, H., Liu, Y., Liu, C., Liu, W. and Li, W. (2021) 'Fault Diagnosis with Spacecraft High-Dimensional Data Based on Machine Learning', Fault Detection, Supervision, and Safety for Technical Processes.

[5] Mukhachev, P. A., Sadretdinov, T. R., Pritykin, D. A., Ivanov, A. B. and Solov'ev, S. V. (2021) 'Modern Machine Learning Methods for Telemetry-Based Spacecraft Health Monitoring', Automation and Remote Control, Vol.82, No.8.

[6] Yu, X., et al. (2020) 'Two-Level Data Compression using Machine Learning in Time Series Database', IEEE ICDE.

[7] Levenets, A. V. (2018) 'Basic Principles and Methods of the System Approach to Compression of Telemetry Data', J. Phys.: Conf. Ser. 944 012074

[8] Hung, N. Q. V., Jeung, H. and Aberer, K. (2013) 'An Evaluation of Model-Based Approaches to Sensor Data Compression', IEEE Transactions on Knowledge and Data Engineering, Vol. 25. No. 11.

[9] R. Logeswaran, C. Eswaran (2001), 'Performance Survey of Several Lossless Compression Algorithms for Telemetry Applications', International Journal of Computers and Applications, 23:1, 1-9

[10] Arcangeli, J.-P., Crochemore, M., Hourcastagnou, J.-N. and Pin, J.-E. (1992) 'Compression for an Effective Management of Telemetry Data', JPL-SpaceOps.

[11] de la Vega-Martínez, M., et al. (2021) 'Implementation of the cFS Framework for the Development of Software in Aerospace Missions: First Application in an Undergraduate Program in Mexico', IFAC PapersOnLine.

[12] Song, Y., Lu, Y. (2015), 'Decision tree methods: applications for classification and prediction', Shanghai Archives of Psychiatry, Vol.27, No.2, pp.130-135.

[13] Yamagiwa, S., Hayakawa, E. and Marumo, K. (2020) 'Stream-Based Lossless Data Compression Applying Adaptive Entropy Coding for Hardware-Based Implementation', Algorithms.

[14] Dipperstein, M. (2018) 'Rice (Golomb) Coding Encoding Discussion and Implementation'.

[15] Dipperstein, M. (2018) 'Huffman Code Discussion and Implementation'.

[16] Cudmore, A. (2015) 'The Core Flight System Community: Providing Low Cost Solutions for Small Spacecraft', NASA Technical Report 20150023353.

[17] NASA (2021) Core Flight System (cFS) Training, NASA/TM-20205000691/REV 1, Flight Software Systems Branch, Goddard Space Flight Center, Greenbelt, MD.

[18] McComas, D. (2012) 'NASA/GSFC's Flight Software Core Flight System', Flight Software Workshop, Southwest Research Institute, San Antonio, Texas, November 7-9.

[19] Vishnoi, R., Bowden, M. and Akin, D. L. (2025) 'Deep Learning at the Edge of Space', SBA, Vol.XXX, No.XX, p.9.

[20] McComas, D. (2025) 'cFS Overview', cFS Basecamp-v3.2.

[21] McComas, D. (2025) 'cFS Framework', cFS Basecamp-v3.2.

[22] McComas, D. (2025) 'cFS Basecamp Application Design', cFS Basecamp-v3.2.